

Informe del Espacio de Trabajo

Agente IA de Fabricación Digital

Análisis Automatizado del Repositorio

23 de junio de 2026

Índice

1. Resumen Ejecutivo	2
2. Arquitectura del Sistema	2
2.1. Filosofía Operativa	2
3. Componentes del Repositorio	2
3.1. Directorio Raíz	2
3.2. Directorio <code>.agent/</code>	3
3.3. Directorio <code>execution/</code> (82 scripts)	4
3.4. Directorio <code>directives/</code> (42 archivos)	7
3.5. Otros Directorios	8
4. Infraestructura y Despliegue	8
4.1. Entorno de Ejecución	8
4.2. Optimización de Recursos	9
4.3. Aceleración por GPU	9
5. Sistema de Memoria	9
6. Flujo de Trabajo Típico	9
7. Estado del Proyecto y Próximos Pasos	10
8. Estadísticas del Repositorio	10

1. Resumen Ejecutivo

Este repositorio aloja un **Agente de IA para Fabricación Digital**, accesible a través de un bot de Telegram. El agente está diseñado como un asistente de ingeniería que puede interpretar lenguaje natural e imágenes para generar archivos de diseño y fabricación. El núcleo del sistema utiliza un LLM para la orquestación y scripts deterministas en un entorno Docker para ejecutar tareas complejas de forma fiable, incluyendo el uso de software CAD/ECAD como **FreeCAD** y **KiCad** en modo headless.

El proyecto se encuentra en desarrollo activo con funcionalidades estables de generación de modelos 3D y PCBs.

2. Arquitectura del Sistema

El sistema sigue una **arquitectura de 3 capas** diseñada para separar la lógica probabilística del LLM de la ejecución determinista del código:

1. **Layer 1: Directivas (directives/)** — Archivos YAML que actúan como SOPs (Procedimientos Operativos Estándar). Definen qué hacer: objetivo, entradas requeridas, pasos, salidas esperadas y casos límite.
2. **Layer 2: Orquestación (LLM)** — El modelo de lenguaje actúa como puente entre la intención humana y la ejecución técnica. Lee directivas, elige herramientas apropiadas, ejecuta flujos multietapa, valida entradas/salidas y gestiona errores.
3. **Layer 3: Ejecución (execution/)** — Scripts deterministas en Python, cada uno con una sola responsabilidad. Entradas por CLI arguments, secretos en `.env`, salidas por stdout (preferiblemente JSON), códigos de salida categorizados (0 = éxito, 1+ = fallo).

2.1. Filosofía Operativa

El sistema opera bajo principios fundamentales:

- **Reuse before building:** Verificar `execution/` antes de escribir un nuevo script.
- **Self-annealing:** Los errores son oportunidades de aprendizaje; máximo 3 intentos de recuperación.
- **Update directives as you learn:** Cada hallazgo se registra sin sobrescribir historia.
- **Validate before moving on:** Confirmar esquema, cantidades, archivos y tiempos después de cada paso.

3. Componentes del Repositorio

3.1. Directorio Raíz

Archivo/Directorio	Propósito
<code>bot.py</code>	Punto de entrada principal del bot de Telegram; carga variables de entorno y delega en <code>bot_manager.py</code> .
<code>README.md</code>	Documentación principal del proyecto en español.
<code>README_inglés.md</code>	Documentación del framework (Gemini Agent Framework) en inglés.
<code>Agente-IA-CIRCUITO-IMPRESO.md</code>	Documento “ADN del Proyecto” que extrae la lógica técnica y componentes esenciales para replicar la automatización.
<code>CHANGELOG.md</code>	Historial de versiones del proyecto (formato Keep a Changelog, SemVer).
<code>TODO.md</code>	Lista de mejoras sugeridas: refactorización del bot, extracción estructurada de JSON, base de datos para estado/memoria, y optimización Docker.
<code>CONTRIBUTING.md</code>	Guía de contribución: GitHub Flow, estándares de directivas y scripts, validación local.
<code>setup.sh</code>	Script de configuración inicial: crea entorno Conda (<code>pcb.env</code>) e instala dependencias Python.
<code>requirements.txt</code>	Dependencias Python del orquestador: <code>pyserial</code> , <code>pathfinding</code> , <code>pcb-tools-extension</code> , <code>python-telegram-bot</code> , <code>chromadb</code> , <code>google-generativeai</code> , <code>duckduckgo-search</code> , <code>opencv-python</code> , entre otras.
<code>Dockerfile.sandbox</code>	Imagen Docker basada en <code>ubuntu:22.04</code> con KiCad 8.0, FreeCAD, <code>pcb2gcode</code> , <code>Xvfb</code> , y módulos Python para CAD/PCB.
<code>.editorconfig</code>	Configuración de estilo de código (UTF-8, LF, indentación 4 espacios, YAML a 2 espacios).
<code>.gitignore</code>	Ignora <code>--pycache--</code> , entornos virtuales, <code>.env</code> , <code>.tmp/</code> , <code>.vscode/</code> , <code>.idea/</code> .
<code>LICENSE</code>	Licencia MIT.
<code>memoria.tex</code>	Documento LaTeX sobre la implementación de memoria persistente en agentes de IA con ChromaDB.
<code>PCB.png</code>	Imagen representativa del proyecto (PCB renderizada).

3.2. Directorio `.agent/`

Contiene las instrucciones de sistema y contexto del agente:

- `AGENT_FRAMEWORK.md`: Define la arquitectura de 3 capas, el modelo mental del agente como middleware entre intención e implementación, principios operativos, protocolo de notificaciones y organización de archivos.
- `AGENT_INSTRUCTIONS.md`: Instrucciones detalladas de sistema para el comportamiento del agente.
- `AGENT_FRAMEWORK.md`: Documento marco con guías de setup, lista de directivas disponibles, y principios de operación.

3.3. Directorio execution/ (82 scripts)

Este directorio contiene todos los scripts deterministas de Python. Se organizan en las siguientes categorías funcionales:

CAD 3D (FreeCAD)

- `generate_freecad_script.py`: Genera modelos 3D paramétricos (cubo, cilindro, esfera, cono, engranaje) desde descripciones JSON. Exporta a STL, STEP, OBJ y render PNG. Calcula volumen y masa estimada.
- `render_stl.py`: Renderiza archivos STL a imagen PNG.

Diseño Electrónico (KiCad)

- `generate_kicad_pcb_script.py`: Genera scripts Python para KiCad PCB Editor que crean archivos `.kicad_pcb` con componentes colocados y conexiones definidas. Implementa estrategias de placement por afinidad de red y estrategia SoC (periférico).
- `render_pcb.py`: Renderiza el diseño de PCB a imagen.
- `render_sch.py`: Renderiza el esquemático a imagen.
- `json_to_kicad_netlist.py`: Convierte JSON de diseño a netlist de KiCad.

Fabricación PCB (Gerber/CNC)

- `img_to_gerber.py`: Convierte imágenes blanco y negro a formato Gerber RS-274X.
- `img_to_drill.py`: Detecta taladros en imágenes mediante OpenCV y genera archivos Excellon (`.drl`).
- `generate_gerbers.py`: Genera archivos Gerber completos desde proyectos KiCad.
- `generate_gcode.py`: Convierte archivos Gerber a G-Code para fresado CNC mediante `pcb2gcode`. Parámetros configurables: diámetro de broca, número de pasadas, profundidad de corte, feedrate.
- `generate_simple_gcode.py`: Generación simplificada de G-Code.
- `path_to_gcode.py`: Convierte rutas de herramientas a G-Code.
- `img_to_gcode.py`: Conversión directa de imagen a G-Code.
- `create_manufacturing_zip.py`: Empaqueta Gerbers y Drills en ZIP compatible con JLCPCB/PCBWay.
- `visualize_gcode.py`: Visualización de trayectorias G-Code.
- `generate_test_pattern.py`: Genera patrones de prueba para calibrar detección de taladros.

Comunicación CNC

- `send_gcode.py`: Transmite G-Code a máquinas CNC con firmware GRBL vía puerto serial. Implementa handshaking línea por línea con respuesta `ok`.
- `monitor_esp32.py`: Monitoreo de comunicación con ESP32.
- `flash_esp32.py`: Flasheo de firmware a ESP32.
- `check_serial_ports.py`: Detecta puertos serie disponibles.

Telegram Bot

- `listen_telegram.py`: Bucle principal que escucha mensajes de Telegram (162 líneas). Inicializa SQLite, configura menú de comandos, delega en `telegram_handlers/`.
- `listen_telegram_helpers.py`: Funciones auxiliares: gestión de usuarios, personas, recordatorios, ejecución de herramientas.
- `telegram_tool.py`: Herramienta de envío/recepción de mensajes Telegram.
- `bot_manager.py`: Implementación asíncrona con `python-telegram-bot` (Application, CommandHandler).
- `telegram_handlers/`: Directorio para handlers (actualmente vacío).

LLM y Chat

- `chat_with_llm.py`: Orquestador principal de LLM (501 líneas). Soporta múltiples proveedores con orden de prioridad: Groq (principal), Gemini (fallback), seguido de OpenRouter, OpenAI, Anthropic, DeepSeek y Ollama. Implementa RAG con ChromaDB, compresión de historial (Soft Cap 12 mensajes, Hard Cap 30000 caracteres), limpieza de respuestas markdown.
- `chat_ollama.py`: Interfaz opcional con Ollama para modelos locales (no utilizada actualmente por falta de GPU).
- `chat_openrouter.py`: Conector para OpenRouter (acceso a múltiples modelos).
- `run_agent.py`: Interfaz CLI principal del agente.
- `benchmark_models.py`: Benchmarking de modelos de lenguaje.
- `research_topic.py`: Investigación automática en internet.

Memoria Vectorial (ChromaDB)

- `save_memory.py`: Guarda fragmentos de información en ChromaDB con metadatos (categoría, timestamp).
- `query_memory.py`: Consulta semántica a ChromaDB con puntuación de relevancia.
- `list_memories.py`: Lista recuerdos recientes almacenados.
- `delete_memory.py`: Elimina recuerdos por ID.
- `poc_memory_chroma.py`: Prueba de concepto del sistema de memoria.

Base de Datos y Estado

- `db_manager.py`: Gestor de base de datos SQLite con tablas para usuarios, recordatorios e historial de chat.
- `chat_history.py`: Gestión del historial de conversaciones.

Análisis de Imágenes

- `analyze_circuit_drawing.py`: Analiza fotos de circuitos dibujados a mano mediante LLM + visión, genera netlist JSON.
- `analyze_image.py`: Análisis general de imágenes con IA.
- `test_opencv_vision.py`: Pruebas de visión computacional con OpenCV.

Utilidades y Mantenimiento

- `check_system_health.py`: Validación completa del entorno (Python, .env, dependencias, Docker).
- `check_dependencies.py`: Verificación de dependencias instaladas.
- `check_tool_versions.py`: Verificación de versiones de herramientas del sistema.
- `monitor_resources.py`: Monitoreo de recursos del sistema (RAM, CPU, ZRAM).
- `format_code.py`: Formateo automático de código Python con autopep8.
- `audit_codebase.py`: Auditoría de estructura del código base.
- `generate_tests.py`: Generación automática de pruebas.
- `run_tests.py`: Ejecutor de pruebas.
- `test_check_dependencies.py`: Tests para el verificador de dependencias.
- `test_gemini_connection.py`: Tests de conectividad con API Gemini.
- `pre_commit_check.py`: Validaciones pre-commit.
- `clean_project.py`: Limpieza de archivos temporales y artefactos.
- `clean_out.sh`: Script shell para limpiar directorios de salida.
- `freeze_requirements.py`: Congela dependencias actuales en requirements.txt.
- `update_dependencies.py`: Actualizador de dependencias.
- `backup_project.py`: Creación de respaldos del proyecto.
- `deploy_to_github.py`: Automatización git add/commit/push con versionado semántico.
- `init_project.py`: Inicialización y limpieza de nuevos proyectos desde plantilla.

- `clone_repo.py`: Clonado de repositorios.
- `update_framework.py`: Actualización del framework desde plantilla remota (checkout selectivo).
- `update_from_template.py`: Actualización desde repositorio plantilla original.
- `build_sandbox.py`: Construcción de la imagen Docker sandbox.
- `run_sandbox.py`: Ejecución de comandos dentro del sandbox Docker.
- `scaffold_directive.py`: Generación de esqueleto de nuevas directivas.
- `validate_directives.py`: Validación de sintaxis YAML de directivas.
- `list_directives.py`: Listado de directivas disponibles.
- `auto_document.py`: Documentación automática del proyecto.
- `generate_readme.py`: Generación de README.md mediante LLM.
- `summarize_project.py`: Resumen del proyecto.
- `explain_code.py`: Explicación de fragmentos de código.
- `translate_text.py`: Traducción de texto entre idiomas.
- `text_to_speech.py`: Conversión de texto a voz (gTTS).
- `transcribe_audio.py`: Transcripción de audio a texto.
- `voice_interface.py`: Interfaz de voz completa.
- `scrape_single_site.py`: Extracción de contenido de URLs.
- `alert_user.py`: Alertas audibles de sistema.
- `env_diagnostic.py`: Diagnóstico de variables de entorno (en raíz).
- `list_directory_contents.py`: Listado de contenido de directorios.
- `list_gemini_models.py`: Listado de modelos Gemini disponibles.
- `design_from_text.py`: Diseño desde descripción textual.

3.4. Directorio directives/ (42 archivos)

Contiene los SOPs en formato YAML que definen los flujos de trabajo del agente. Cada directiva incluye: `goal`, `required_inputs`, `steps`, `expected_outputs` y `edge_cases`.

Directivas principales:

- `generate_freecad_script.yaml`: Generación de modelos 3D con FreeCAD.
- `generate_kicad_pcb_script.yaml`: Diseño de PCB automatizado con KiCad.
- `analyze_image.yaml`: Análisis de imágenes con IA.

- `research_topic.yaml`: Investigación en internet.
- `save_memory.yaml` / `query_memory.yaml` / `delete_memory.yaml`: Gestión de memoria vectorial.
- `scrape_website.yaml`: Extracción de contenido web.
- `system_maintenance.yaml`: Mantenimiento del sistema.
- `telegram_remote_control.yaml`: Control remoto por Telegram.
- `voice_interface.yaml`: Interfaz de voz.
- `translate_text.yaml`: Traducción de textos.
- `deploy_to_github.yaml`: Despliegue a GitHub.
- `update_framework.yaml`: Actualización del framework.
- `onboarding.yaml`: Proceso de inducción para nuevos desarrolladores.
- `messages.py`: Mensajes predefinidos para el bot.

3.5. Otros Directorios

Directorio	Propósito
<code>.tmp/</code>	Archivos temporales regenerables: base de datos SQLite (<code>agent_database.db</code>), ChromaDB vector store (<code>chroma_db/</code>), estado de ejecución (<code>run_state.json</code>), archivos de sesión Telegram, salidas de procesos.
<code>.out/</code>	Directorio de salida para artefactos generados.
<code>data/</code>	Datos estáticos del proyecto.
<code>docs/</code>	Documentación adicional: guías (<code>CNC.md</code> , <code>GUIA_ESTUDIANTES_PCB.md</code>), PDFs de explicaciones, reportes de progreso, notas de release v1.0, referencia de comandos.
<code>telegram_handlers/</code> — Directorio para handlers del bot de Telegram (actualmente vacío, preparado para refactorización).	
<code>.vscode/</code>	Configuración del editor VS Code.

4. Infraestructura y Despliegue

4.1. Entorno de Ejecución

El sistema utiliza una arquitectura de dos niveles:

1. **Host:** Python 3.10+ con Conda (`pcb_env`) ejecuta el orquestador y el bot de Telegram.

2. **Sandbox Docker:** Contenedor basado en `ubuntu:22.04` con KiCad 8.0, FreeCAD, `pcb2gcode`, `Xvfb` y dependencias Python para CAD/PCB. Volumen mapeado en `/mnt/out` para persistencia.

4.2. Optimización de Recursos

Para hardware limitado, el sistema está diseñado para poder utilizar **ZRAM** con algoritmo `zstd` (50-60 % de RAM física, `swappiness=150`) en caso de requerir la ejecución de modelos locales (vía Ollama) junto con KiCad/FreeCAD sin agotar la memoria. No obstante, al no contar con GPU dedicada, actualmente el sistema opera principalmente con modelos basados en la nube (APIs como Groq, Google Gemini, etc.), reduciendo así la carga local de almacenamiento y memoria.

4.3. Aceleración por GPU

El agente está preparado para escalar a hardware con GPU mediante `nvidia-container-toolkit`. Esto permitiría en el futuro desplegar y ejecutar modelos locales como `gemma2:9b` o `llama3` con latencia reducida si se dispone de dicho hardware.

5. Sistema de Memoria

El agente implementa una memoria a largo plazo (LTM) mediante:

- **ChromaDB:** Base de datos vectorial local en `.tmp/chroma_db/`. Almacena aprendizajes, soluciones a errores y preferencias del usuario con metadatos (categoría, timestamp).
- **RAG (Retrieval Augmented Generation):** El orquestador `chat_with_llm.py` consulta automáticamente ChromaDB para inyectar contexto relevante en las respuestas del LLM.
- **SQLite:** Base de datos relacional en `.tmp/agent_database.db` para usuarios, recordatorios e historial de chat.

6. Flujo de Trabajo Típico

1. **Entrada:** Usuario envía foto de un circuito dibujado a mano + descripción de función vía Telegram.
2. **Análisis:** `analyze_circuit_drawing.py` usa LLM con visión para identificar componentes y generar netlist JSON.
3. **Diseño:** `generate_kicad_pcb_script.py` crea script de KiCad con placement y conexiones.
4. **Fabricación:** `generate_gerbers.py` produce archivos Gerber; `img_to_drill.py` detecta taladros; `create_manufacturing_zip.py` empaqueta todo para el fabricante.
5. **CNC:** `generate_gcode.py` convierte Gerbers a G-Code; `send_gcode.py` transmite a la máquina CNC por puerto serial.

7. Estado del Proyecto y Próximos Pasos

Estado: Desarrollo activo. Funcionalidades principales de generación 3D y PCB son estables.

Mejoras planificadas (de TODO.md):

1. Refactorización de `listen_telegram.py` (monolito de 1300+ líneas) a arquitectura basada en handlers.
2. Extracción estructurada de JSON con Pydantic o Gemini Schema para evitar reintentos de parsing.
3. Migración de almacenamiento plano en `.tmp/` a SQLite para consistencia de datos.
4. Optimización de Docker con multi-stage builds para reducir tiempo de reconstrucción.

8. Estadísticas del Repositorio

- Total de scripts Python en `execution/`: 82
- Total de directivas YAML en `directives/`: 41
- Total de archivos en el repositorio: 31 entradas en raíz
- Licencia: MIT
- Versionado: SemVer con changelog automatizado